DENSITY ESTIMATION USING REAL NVP

Course Name: [EE6180 Advanced topics in artificial intelligence]
Student Name: Palika Charitha
Student Roll Number: BT22D109

---

1 Contributions of the chosen paper

- The paper introduces a real-valued non-volume preserving (real NVP) transformations to maximize exact log likelihood unlike other generative methods which approximate the log likelihood estimation.
- In this model we start with a prior distribution and transform it in a sequential manner to result in true distribution. Assume x is the data from data distribution $p_x(x)$ and z is the latent variable from prior distribution $P_z(z)$. An invertible function maps from z to x. Then the distribution over x can be written as:

$$p_X(x) = p_Z(f(x)) \left| \det \left(\frac{\partial f(x)}{\partial x^T} \right) \right|$$

  To maximize likelihood of the above equation we need to compute determinant of Jacobian at each point which is computationally intensive. The authors propose invertible transformations using affine coupling layers such that the Jacobian is a triangular matrix, thereby reducing the computational complexity in computing the determinants. To ensure simpler computation for determinant of Jacobian they use masked convolution in these layers where a binary mask is applied to the image before feeding into the coupling layers
- As they mentioned in discussions, this model tries bridge gaps between the generative models. Similar to auto-regressive models it achieves tractable, exact log-likelihood evaluation and similar to VAEs it allows more flexible form leading to faster sampling.
- Unlike VAE's they try to learn more semantically meaningful latent spaces which is key feature for representation learning.

2 Reproduced Experiments

2.1 Experiments on simple 2D distributions

Initial experiments were conducted to generate samples from simpler distributions such as Two Moons, circular gaussian etc. These experiments were run on colab.

2.1.1 Training details

Component	**Description**
Input Distribution	2D Gaussian $\mathcal{N}(0, I)$
Model	Real NVP with 32 affine coupling layers
Loss Function	Negative Log-Likelihood
Optimizer	Adam
Iterations	4000- 10000

Table 1: 2D Gaussian to Two Moons using Real NVP

2.1.2 Results

- Two Moon distribution

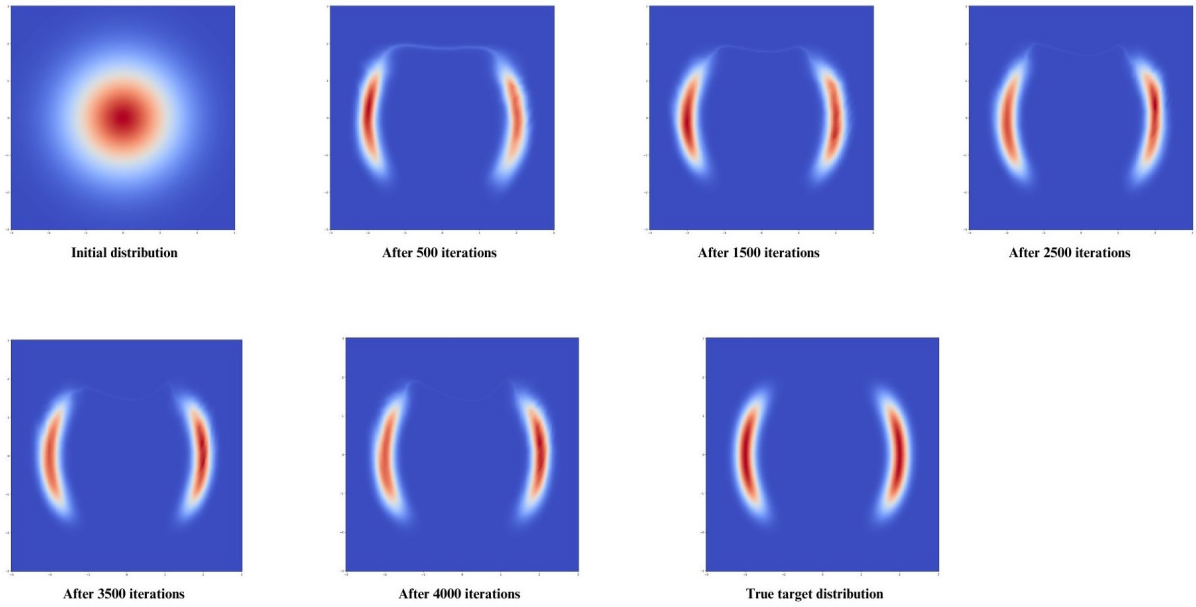


Figure 1: Figure showing the transformation of 2D gaussian distribution to target Two Moons distribution

- Ring dataset

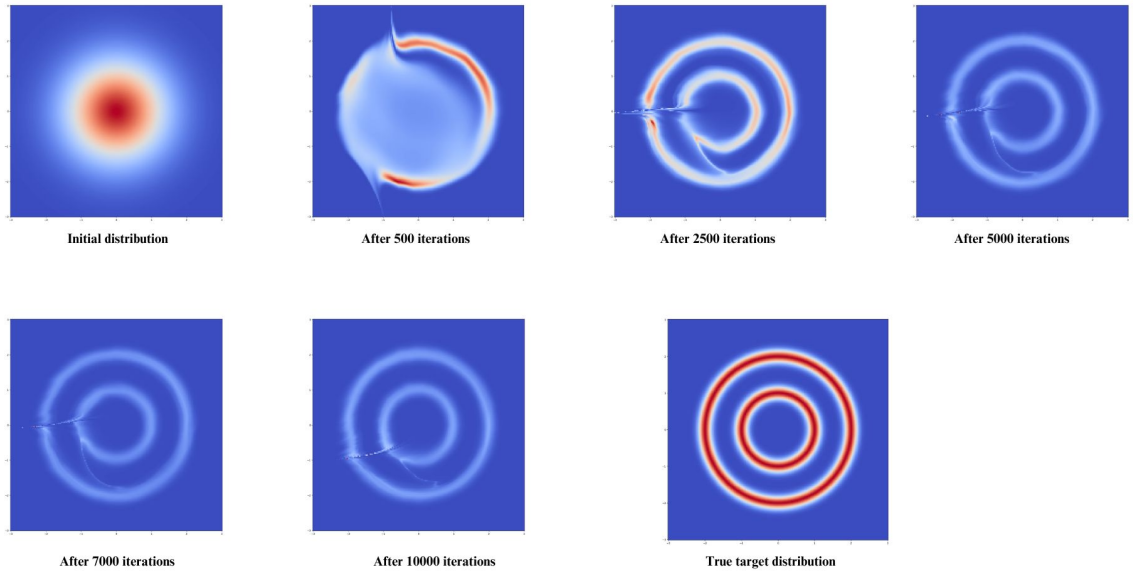


Figure 2: Figure showing the transformation of 2D gaussian distribution to target Two Moons distribution

- Circular Gaussian dataset

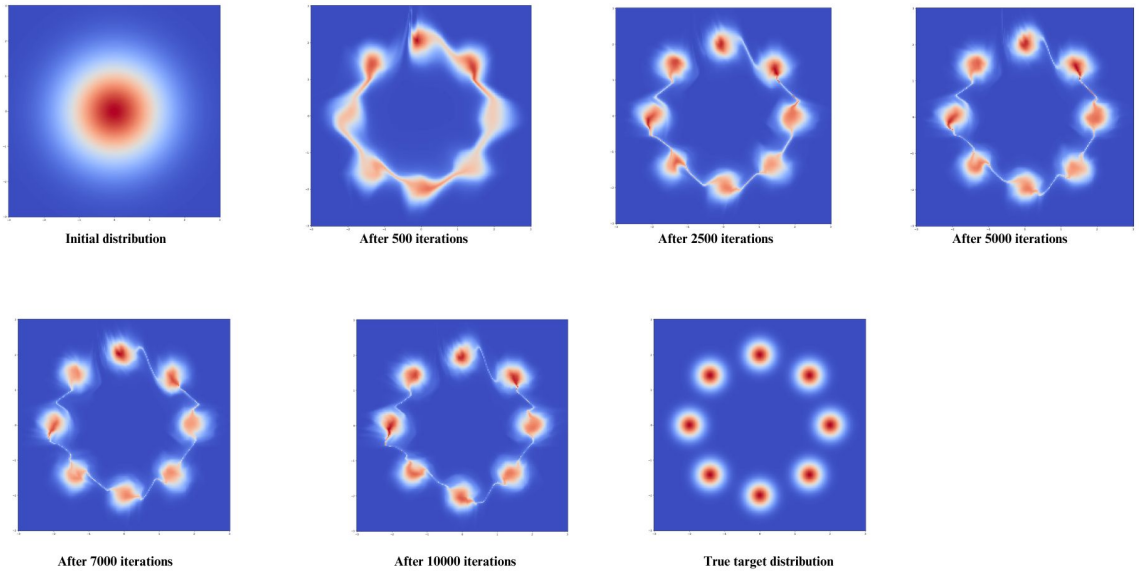


Figure 3: Figure showing the transformation of 2D gaussian distribution to target Circular gaussian distribution

2.2 Experiments on Image datasets

All the experiments on Image datasets are ran on Kaggle.

2.2.1 Dataset and Preprocessing:

Dataset	**Description**	**Weblink**
CIFAR-10	torchvision.dataset	`https://pytorch.org/vision/0.19/generated/torchvision.datasets.CIFAR10.html`
CelebA	Kaggle dataset	`https://www.kaggle.com/datasets/jessicali9530/celeba-dataset`
ImageNet 64	Kaggle dataset	`https://www.kaggle.com/datasets/lijiyu/imagenet`

Table 2: Dataset used

2.2.2 Architecture details

Component	Value / Description
Prior Distribution	Standard Normal $\mathcal{N}(0, I)$
Base Feature Dimension	64
Residual Blocks per Coupling Net	8
Skip Connections	Enabled
Weight Normalization	Enabled
Batch Normalization	Enabled
Coupling Type	Affine (scale + shift)
Batch Size	64
Learning Rate	1e-3
Optimizer	Adamax
weight decay/ L2 penalty	0.99
Max Epochs	10-50 depending on datasets

Table 3: Hyperparameters and Prior Distribution used in RealNVP

2.2.3 Results

Figure 4: Samples generated using the model at different epochs for CIFAR10 dataset

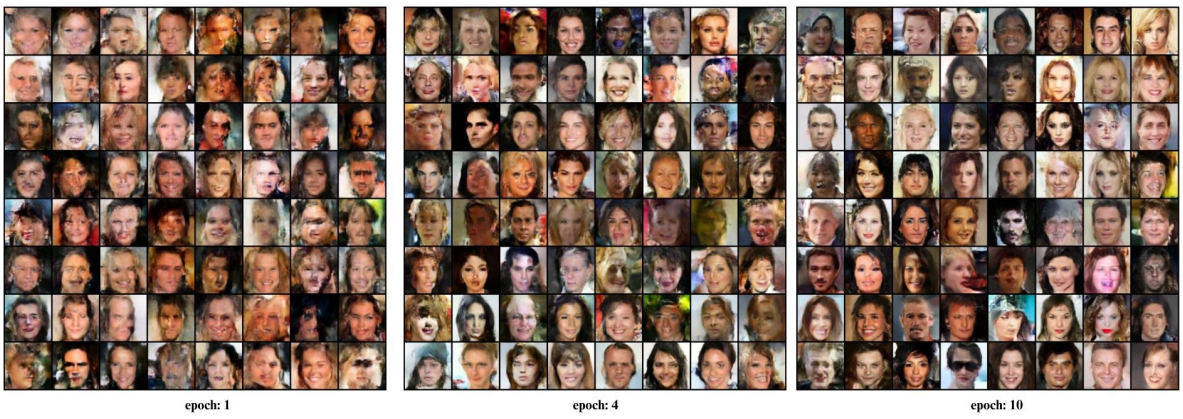


Figure 5: Samples generated using model for different epochs for CelebA dataset

Figure 6: Samples generated using model for different epochs for ImageNet 64 dataset

Dataset	Train bits/Dim	Validation bits/Dim
CIFAR-10	3.6	3.75
CelebA	2.690	22.02
ImageNet64	3.374	3.419

Table 4: Bits per Dimension on Different Datasets

3 New Experiments

3.1 Experiment-1: Additive coupling

3.1.1 Description

Affine coupling layers are replaced with additive coupling layers, similar to the coupling used in paper [1]. Additive coupling layers are less complex and computationally more effective layers. In this, the input is partitioned, and one half of it is used to compute the shift and is added to the other half.
In an additive coupling layer, the input tensor $\mathbf{x} \in R^{B \times C \times H \times W}$ is split into two parts:

$$\mathbf{x} = [\mathbf{x}_A, \mathbf{x}_B]$$

where $\mathbf{x}_A$ represents the part of the input that is unchanged during the transformation, and $\mathbf{x}_B$ is the part that will be transformed. The transformation applied to $\mathbf{x}_B$ is given by:

$$\mathbf{y}_A = \mathbf{x}_A$$

$$\mathbf{y}_B = \mathbf{x}_B + t(\mathbf{x}_A)$$

where $t(\cdot)$ is a neural network that computes a shift applied to $\mathbf{x}_B$ based on $\mathbf{x}_A$. The output tensor is:

$$\mathbf{y} = [\mathbf{y}_A, \mathbf{y}_B]$$

This transformation is invertible and the inverse of the transformation is given by:

$$\mathbf{x}_A = \mathbf{y}_A$$

$$\mathbf{x}_B = \mathbf{y}_B - t(\mathbf{y}_A)$$

Since the transformation is additive, the Jacobian determinant is 1, which implies that no volume change occurs during the transformation. This simplifies the computation of the log-likelihood.

3.1.2 Results

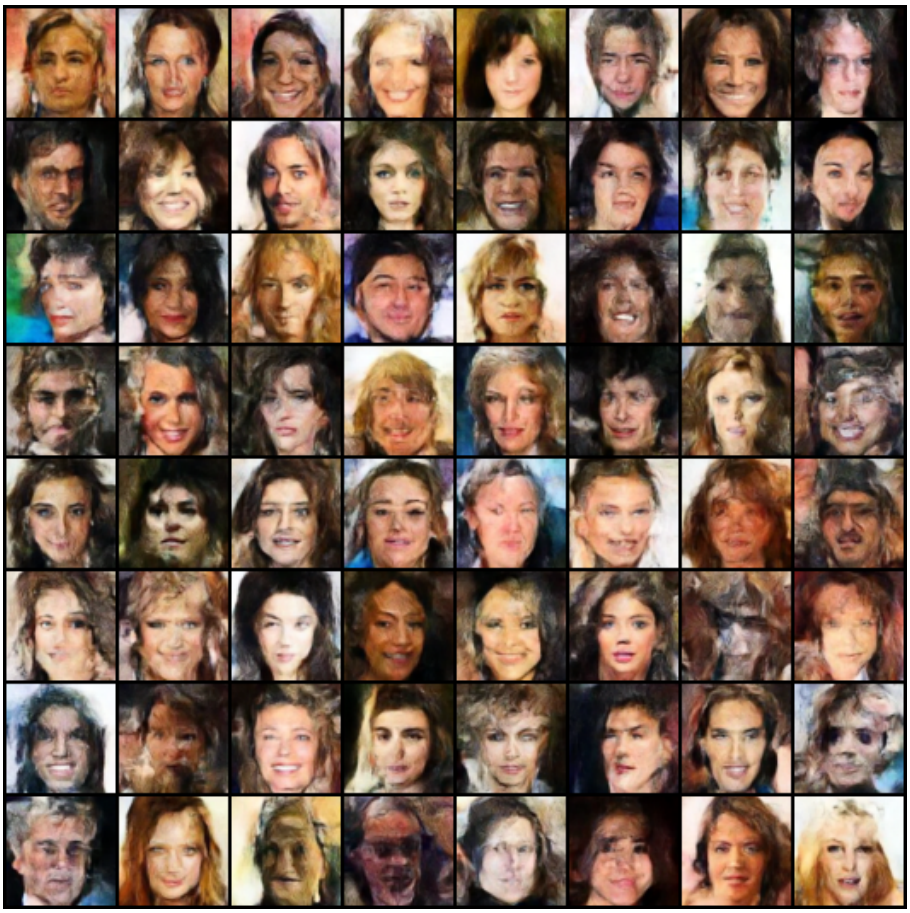


Figure 7: Fig: Samples generated after 10 epochs of training

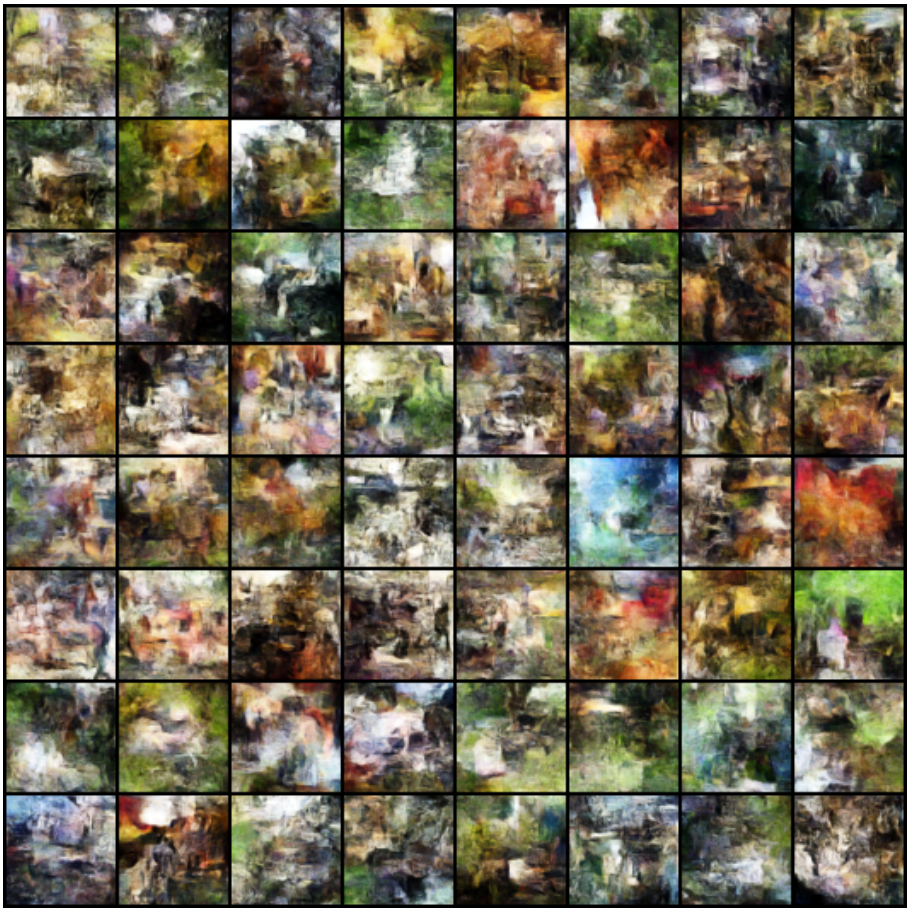


Figure 8: Samples generated after 10 epochs of training

3.2 Experiment-2: CBAM (Convolutional Block Attention Module)

3.2.1 Description

Instead of directly passing the input into the affine coupling block after batch normalisation, the input image is first processed through a Convolutional Block Attention Module (CBAM) to enhance its representation [2]. This enables the model to focus on important features and improve the quality of the learned transformations by incorporating spatial and channel-wise attention mechanisms.

1. Channel Attention Module (CAM)

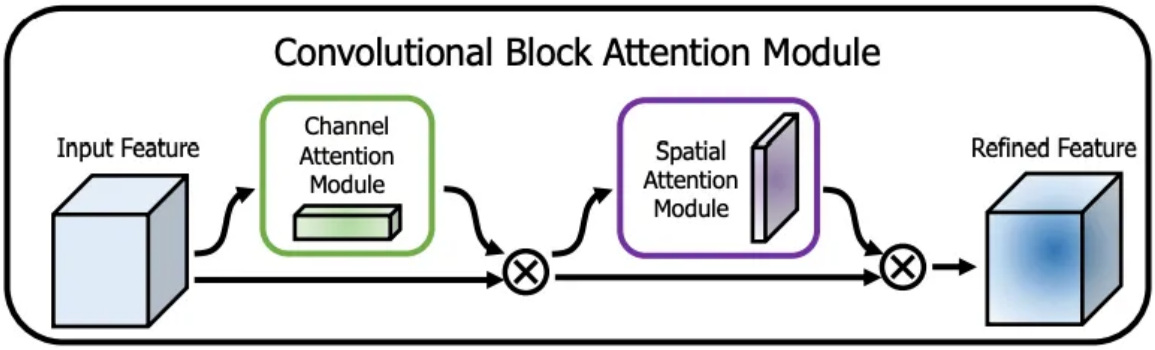


Figure 9: CBAM (Convolutional Block Attention module (link)

The goal of the channel attention module is to compute the importance of each channel in the feature map $\mathbf{F} \in R^{H \times W \times C}$

Step 1: Global Pooling: Applies global max and average pooling over each channel

$$\mathbf{F}_{avg} = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \mathbf{F}(i,j)$$

$$\mathbf{F}_{max} = \max_{i,j}(\mathbf{F}(i,j))$$

Step 2: Channel Attention Mechanism: These two pooled features are passed through two fully connected layers to generate the channel-wise attention

$$\mathbf{F}'_{avg} = \mathrm{FC}_1(\mathrm{ReLU}(\mathrm{FC}_0(\mathbf{F}_{avg})))$$

$$\mathbf{F}'_{max} = \mathrm{FC}_1(\mathrm{ReLU}(\mathrm{FC}_0(\mathbf{F}_{max})))$$

Step 3: Channel Attention Output: The final channel attention map is obtained by adding the results of the two streams and applying a sigmoid activation:

$$\mathbf{M}_c = \sigma\left(\mathbf{F}'_{avg} + \mathbf{F}'_{max}\right)$$

Thus, the output feature map after applying the channel attention is:

$$\mathbf{F}_c = \mathbf{M}_c \odot \mathbf{F}$$

2. Spatial Attention Module (SAM)

The spatial attention module focuses on the importance of different spatial locations in the feature map.
Step 1: Channel-Wise Pooling: First, apply channel-wise pooling (either average or max) across the feature map to get a 2D map representing spatial attention:

$$\mathbf{F}_{avg}^{spatial} = \frac{1}{C} \sum_{k=1}^{C} \mathbf{F}(i,j,k)$$

$$\mathbf{F}_{max}^{spatial} = \max_{k} \mathbf{F}(i,j,k)$$

Step 2: Concatenation of Pooling Results The results from both average and max pooling are concatenated along the channel dimension:

$$\mathbf{F}_{concat} = \mathrm{concat}(\mathbf{F}_{avg}^{spatial}, \mathbf{F}_{max}^{spatial})$$

Step 3: Spatial Attention Mechanism The concatenated result is passed through a 7×7 convolution to generate the spatial attention map:

$$\mathbf{M}_s = \sigma\left(\mathrm{Conv}(\mathbf{F}_{concat})\right)$$

Thus, the final spatial attention applied to the input feature map is:

$$\mathbf{F}_s = \mathbf{M}_s \odot \mathbf{F}$$

3. Final Output of CBAM The final output of the CBAM module is obtained by sequentially applying the channel and spatial attention modules. This output is then passed to subsequent layers for further processing.

3.2.2 Results

Figure 10: Samples generated after 10 epochs of training

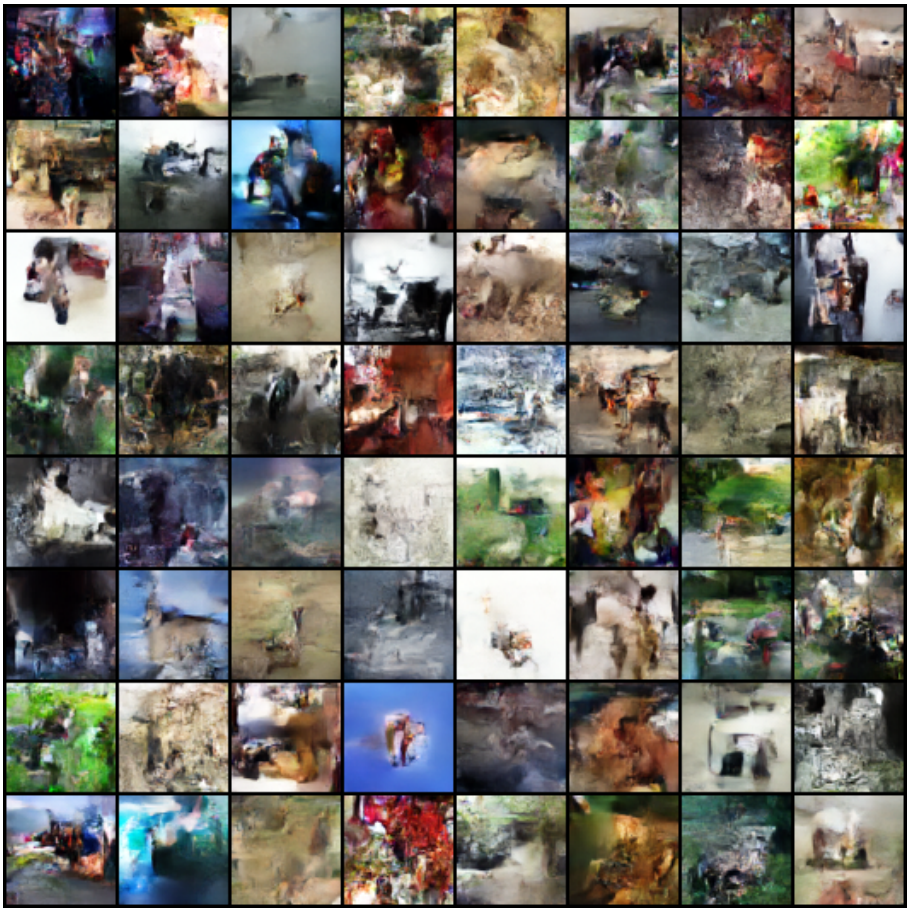


Figure 11: Samples generated after 10 epochs of training

3.3 Experiment-3: Attention-based coupling

3.3.1 Description

Instead of affine coupling, cross-attention-based coupling was implemented where:

- The masked part of the input serves as the context.
- The unmasked part of the input is used as the query.

The output of the cross-attention layer is further processed by successive convolution layers to generate the shifting and scaling vectors for transforming the unmasked part of the input.

Cross-Attention Coupling Layer

In the cross-attention coupling layer, the input tensor $\mathbf{x} \in R^{B\times C\times H\times W}$ is split into two parts:

$$\mathbf{x} = [\mathbf{x}_A, \mathbf{x}_B]$$

where $\mathbf{x}_A$ is the masked part and $\mathbf{x}_B$ is the unmasked part. The goal is to apply a cross-attention mechanism between these parts.

Step 1: Cross-Attention We use a cross-attention mechanism where $\mathbf{x}_A$ serves as the context and $\mathbf{x}_B$ serves as the query. The attention is computed using the scaled dot-product attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

where: - $Q = \mathbf{x}_B$ (unmasked part), - $K = \mathbf{x}_A$ (masked part), - $V = \mathbf{x}_A$ (masked part, used as value).

Thus, the output of the attention mechanism is:

$$\mathbf{z} = \text{Attention}(\mathbf{x}_B, \mathbf{x}_A, \mathbf{x}_A)$$

Step 2: Shift and Log-Scale Generation The output $\mathbf{z}$ is passed through a convolutional layer to generate the shift and log-scale for transforming $\mathbf{x}_B$:

$$\textbf{shift}, \textbf{log\_scale} = \text{Conv}(\mathbf{z})$$

Step 3: Apply Transformation The final transformation applied to $\mathbf{x}_B$ is:

$$\mathbf{y}_B = \mathbf{x}_B \odot \exp(\textbf{log\_scale}) + \textbf{shift}$$

where $\odot$ denotes element-wise multiplication.
Step 4: Combine the Parts Finally, we recombine the transformed $\mathbf{y}_B$ with $\mathbf{x}_A$ to obtain the final output:

$$\mathbf{y} = [\mathbf{x}_A, \mathbf{y}_B]$$

3.3.2 Results

Figure 12: Samples generated after 10 epochs

3.4 Comparative analysis

The table below compares different experiments across several datasets in terms of bits per dimension (bits/dim).

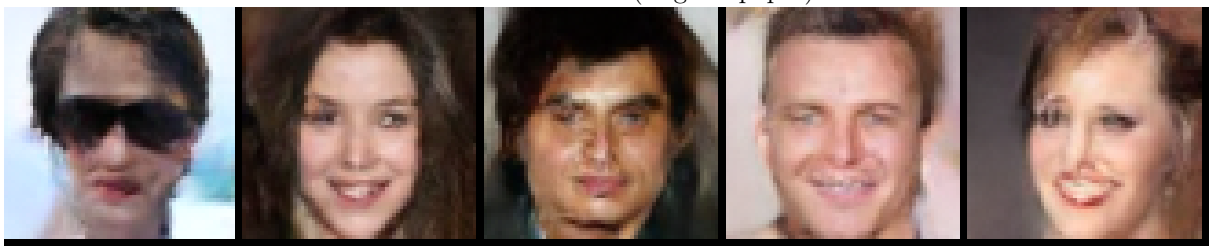
Figure 13: Samples generated after 10 peochs

Experiment	Dataset	bits/dim
Affine Coupling (Original Paper)	CelebA	2.98
Additive Coupling	CelebA	3.015
Affine with CBAM	CelebA	2.645
Affine with Attention	CelebA	3.12
Affine Coupling (Original Paper)	ImageNet	3.419
Additive Coupling	ImageNet	4.103
Affine with CBAM	ImageNet	3.234
Affine with Attention	ImageNet	4.301

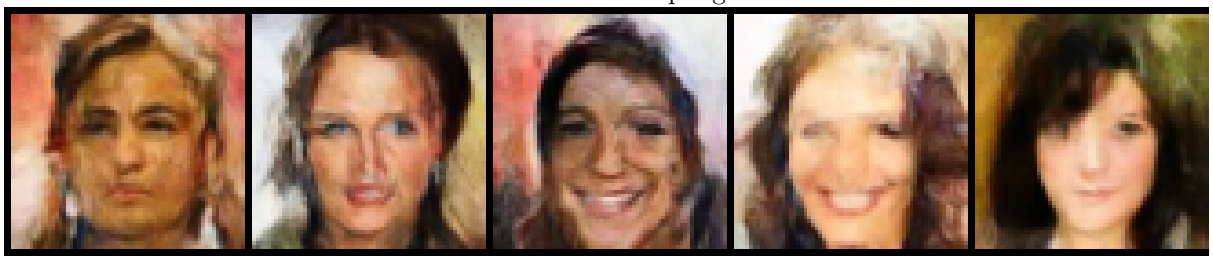
Table 5: Comparison of Coupling Methods and Datasets in terms of Bits/Dim

Best generated images via manual inspection for all the experiments for celebA dataset:

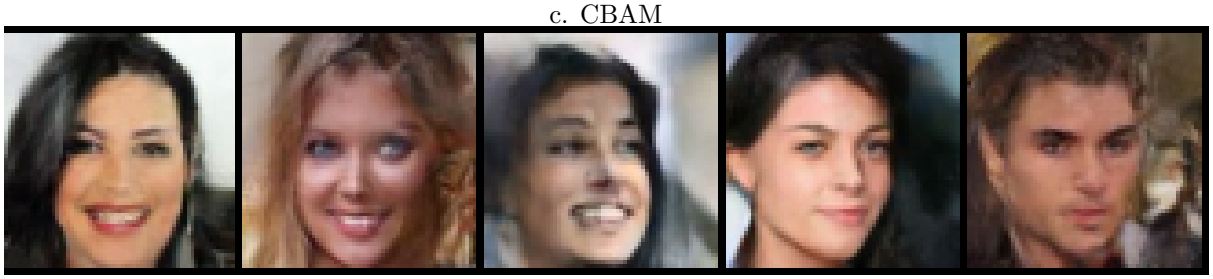
a. Real NVP results(original paper)



b. Additive coupling



c. CBAM



d. Cross attention

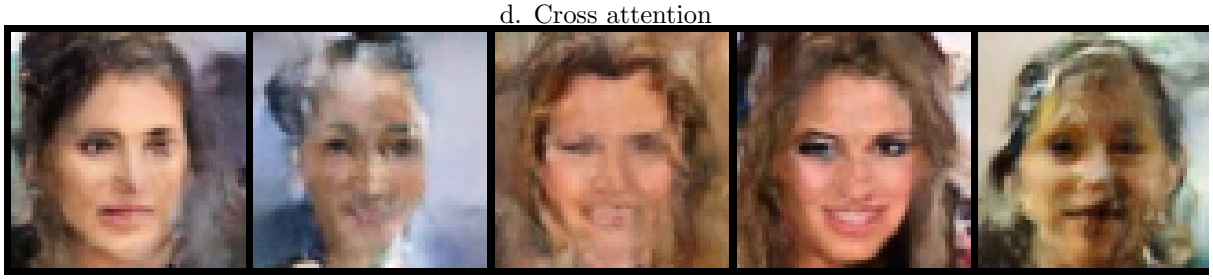


Figure 14: 4x1 Grid of Numbered Images

Via manual inspection, the results from CBAM and the original NVP model perform better than

other models. The same can also be observed from bits/dim.

4 Criticism

1. Training epochs: Few datasets showed higher validation bits/dim, indicating possible underfitting or insufficient training. The report mentions only 10–50 epochs and may need more training epochs to get better bits/dim
2. Sampling Quality: While visual samples were shown, no perceptual quality metrics (e.g., FID, Inception Score) were reported.
3. Affine coupling: Since only part of the input is transformed at a time (the other half remains unchanged), the model may require many coupling layers to capture complex dependencies. The scale and shift functions in affine coupling often rely on convolutional networks, thereby limiting the transformation capacity.
4. Additive Coupling: Removing the scaling factor reduces expressiveness, leading to poorer performance i.e. higher bits/dim
5. CBAM: Attention modules enhance the feature learning but increase the training complexity
6. Crossattention: Although expected to give the best bits/dim, it gave the highest bits/dim; likely due to architectural mismatch or unstable attention across spatial features.

5 Novel idea and Hypothesis

5.1 Hypothesis

Integrating attention modules (such as CBAM) at multiple points in the RealNVP architecture—both before and within coupling layers—can enhance feature learning, improve bits/dim, and produce higher-quality samples.

5.2 Motivation

Initial experiments showed that:

- CBAM at the input level led to lower bits/dim and improved sample quality.
- Cross-attention inside coupling was conceptually interesting but underperformed, likely due to instability or architectural mismatch.

This suggests that attention is beneficial, but its placement and design matter. A more structured and layered integration may amplify its benefits without disrupting the invertibility or efficiency of RealNVP.

5.3 Proposed future implementation

Hierarchical Attention Integration

- **CBAM Preprocessing (Input Level)**: Continue using CBAM after batch normalisation to enhance salient features early.
- **Attention within affine coupled layers:** Embed CBAM blocks inside the scale and shift networks of affine coupling layers. These subnetworks compute the transformation functions s(x) and t(x), and benefit from enhanced feature selectivity.
- **Cross attention:** Usage of cross attention has to be further explored because the initial training results of it have shown higher bits/dim. Training showed a decrease in bits/dim but at a slower pace. Thus, increasing training epochs or adding normalisation might improve its stability as well.

Please note that the affine coupling mechanism in RealNVP inherently has limited expressivity, as it transforms only a subset of the input at each step to enable efficient Jacobian determinant computation. While incorporating attention modules (such as CBAM) can enhance feature selectivity and improve performance, they alone may not be sufficient to achieve the highest possible sample quality. To reach state-of-the-art generation performance, more expressive flow architectures—such as Residual Flows or Flow Matching methods—may be required. The proposed hypothesis is only to extend the original RealNVP framework by strategically integrating attention to improve image generation quality.

6 Code and Notes

6.1 Code files

Code for the experiments conducted is given in the Link
The link has copies of code for all the different experiments conducted. It is organised into Review-1 and Review-2 subfolders for first and second submissions, respectively. The experiments for image generation were run on Kaggle using Kaggle datasets as mentioned in the above sections.

References

[1] Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.

[2] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: Convolutional block attention module. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.